

CEquality

Scoring de crédit Mobile Money Afrique de l'Ouest

Livrable 1 —Architecture, Risques & Coûts

Master IA · 2iE · ECUE2 Sécurité & Coûts dans le Cloud Juin 2026

01 PRÉSENTATION DU PROJET

CEquality est une application web de scoring de crédit ciblant les utilisateurs du service Wave Mobile Money en Afrique de l'Ouest. Elle évalue la capacité d'emprunt et la santé financière de personnes qui ne disposent pas d'historique bancaire traditionnel, à partir de l'analyse de leurs relevés de transactions Mobile Money.

L'utilisateur importe son relevé PDF Wave. L'application extrait automatiquement les données de transactions, calcule un ensemble de variables financières agrégées et temporelles, puis soumet ces variables à un modèle de machine learning. Un score de crédit est restitué avec un niveau de confiance et une explication synthétique.

Stack technique : backend FastAPI (Python), frontend Angular avec TailwindCSS, extraction PDF via pdfplumber / PyMuPDF, modèle Gradient Boosting Regressor. Les données synthétiques de test ont été générées avec ReportLab pour couvrir quatre profils-types d'utilisateurs.

Utilisateurs cibles : particuliers souhaitant connaître leur score avant une demande de crédit, et institutions de microfinance souhaitant automatiser une première analyse de dossier.

02 SCHÉMA D'ARCHITECTURE PRÉVUE

L'architecture retenue est la variante budget : les conteneurs s'exécutent dans des sous-réseaux publics protégés par des Security Groups, sans NAT Gateway. La base de données et le modèle ML restent en accès strictement privé.

Flux de données :

- Le navigateur de l'utilisateur atteint un **Application Load Balancer** public (HTTPS).
- L'ALB route **/api/*** vers le backend FastAPI et les autres routes vers le frontend React servi par nginx — deux services ECS Fargate distincts.
- Au démarrage, le backend charge le **modèle ML depuis S3** ; les données utilisateur sont lues/écrites en base PostgreSQL (sous-réseau privé).
- Le bucket S3 du modèle et la base de données sont **privés** : aucun accès depuis Internet.

Seul l'ALB est exposé publiquement. Le modèle, la base et toutes les données de relevés sont confinés en zone interne.

03 ANALYSE DE RISQUES

Risque	Impact	Proba.	Mesure prévue
Fuite de secrets dans Git	Élevé	Moyenne	.gitignore + .env.example ; scan trufflehog en CI
Relevé PDF exposé (S3 public)	Élevé	Moyenne	Block Public Access, chiffrement S3, bucket privé
Accès non autorisé à l'API	Moyen	Moyenne	JWT, CORS restreint, HTTPS obligatoire
Extraction du modèle	Élevé	Moyenne	Rate-limit 20 req/min/IP, auth requise sur /predict
Injection via le PDF parsé	Élevé	Faible	Validation Pydantic, sandbox extraction PDF
Image Docker vulnérable	Moyen	Élevée	Scan Trivy en CI, image de base figée
Dérive des coûts cloud	Moyen	Moyenne	Architecture budget sans NAT, alarme AWS Budget
Panne d'une zone	Moyen	Faible	Sous-réseaux sur deux AZ distinctes

Le risque prioritaire de CEquality est la confidentialité des relevés Mobile Money, qui contiennent des données personnelles et financières sensibles. C'est ce qui justifie le confinement strict du stockage en sous-réseau privé et le chiffrement au repos.

04 GESTION DES ACCÈS (IAM)

Rôle	Pour qui	Permissions	Principe
Administrateur	Responsable projet	Accès complet AWS	MFA obligatoire
Développeur	Membres du groupe	ECS, ECR, S3, CloudWatch	IAM
CI/CD	GitHub Actions	Push ECR + déploiement ECS	Identité fédérée OIDC
Rôle exécution ECS	Service ECS	Pull ECR, écriture logs CloudWatch	Rôle technique standard
Rôle applicatif ECS	Code backend	s3:GetObject sur le seul bucket modèle	Moindre privilège strict

Les secrets CI (rôle AWS à assumer) sont stockés dans GitHub Secrets. Les secrets applicatifs (clé JWT, URL de base) sont injectés depuis AWS SSM Parameter Store au démarrage du conteneur. Aucun secret n'est écrit en clair dans le dépôt.

05 ESTIMATION DES COÛTS

Hypothèses : environ 200 utilisateurs par mois, quelques centaines de scores générés par jour, services actifs en continu, architecture budget (conteneurs en sous-réseau public, pas de NAT Gateway).

Poste	Service AWS	Coût mensuel estimé
Backend + Frontend	ECS Fargate, 2 tâches (0.25 vCPU / 512 Mo)	~27 \$
Répartiteur de charge	Application Load Balancer	~18 \$
Base de données	RDS PostgreSQL db.t3.micro	~15 \$
Stockage modèle	S3 Standard (< 50 Mo)	< 1 \$

Registre images	ECR + CloudWatch Logs	~2 \$
NAT Gateway	Évité (architecture budget)	0 \$
Total estimé		~63 \$/mois

L'architecture de référence (sous-réseaux privés) ajouterait un NAT Gateway (~32 \$/mois), portant le total à ~95 \$/mois. La variante budget économise ce poste au prix d'une surface réseau plus large, compensée par des Security Groups restrictifs. Chiffres à recalculer avec l'AWS Pricing Calculator avant remise.

06 PLAN DE MISE EN OEUVRE — LIVRABLE 2

- **.gitignore** complet et **.env.example** sans aucune valeur réelle
- Scans en CI : **trufflehog** (secrets), **pip-audit** (dépendances Python), **Trivy** (image Docker)
- Authentification de l'API : **JWT** avec expiration courte, HTTPS obligatoire
- Rate-limit sur **/api/predict** : 20 requêtes / minute / IP
- **Block Public Access** vérifié et activé sur tous les buckets S3
- Secrets applicatifs déplacés vers **AWS SSM Parameter Store**
- Certificat TLS via **AWS Certificate Manager** sur l'ALB
- Alarme de budget AWS configurée à **80 \$** (seuil d'alerte)